Applied Ensemble Learning: Group Work Plan

EECE 490/690 - Introduction to Machine Learning - AUB - Fall 2026-2027

In short: use **Dataset B (Heart Disease)**, work in **Python on Google Colab**, and split the notebook into three blocks. Each member owns one block from code to video.

Who does what

Member	Responsibility	Parts	Approx. points
Member 1	Data foundation: setup box, EDA, preprocessing pipelines	Setup, 1, 2	35
Member 2	Modeling: Random Forest vs. boosting, SMOTENC	3, 4	30
Member 3	Evaluation, SHAP, conclusions, final notebook integration	5, 6 (bonus), 7	35 + 10 bonus

The work is sequential: Member 2 needs Member 1's preprocessing, and Member 3 needs Member 2's models. Member 1 should therefore deliver a shared setup cell early.

Why Dataset B

- **Better preprocessing story (20 pts).** Cholesterol has many zero values and RestingBP has at least one. A cholesterol of 0 is impossible, so there is a real data-quality decision to justify. Dataset A's missing values are ones you hide yourselves.
- **SMOTENC is more meaningful.** B mixes categorical and numeric features, exactly the case SMOTENC exists for.
- **Clear metric justification.** Missing a heart-disease patient (false negative) is costly, so recall-focused evaluation argues itself.
- **SHAP is easy to interpret.** Features like ST_Slope, ChestPainType and Oldpeak make clinical sense.
- **Fast.** 918 rows means every CV run takes seconds.

Downside: B is roughly balanced (about 55% positive), so SMOTE probably will not help much. The assignment explicitly says that is a valid finding.

Tools

- **Google Colab** with one shared master notebook in Google Drive. Each member develops in their own copy, then pastes the finished section into the master.
- **Libraries:** pandas, numpy, matplotlib / seaborn, scikit-learn, imbalanced-learn, xgboost, shap, lime (optional).
- **Fix one seed** (e.g. `SEED = 42`) and create one `StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)` object that everyone reuses, so comparisons stay fair.

Member 1: Data foundation (Setup box, Part 1, Part 2)

1. Experiment Setup Box (first markdown cell)

- Dataset B; target `HeartDisease`, positive class = 1; 918 rows; seed 42; 80/20 stratified split.
- Primary metric: recall on the positive class, ROC-AUC as secondary, because missed diagnoses are the costliest error.
- Group members and their responsibilities.

2. Part 1 - EDA

- Feature types: numeric (Age, RestingBP, Cholesterol, MaxHR, Oldpeak), categorical (Sex, ChestPainType, RestingECG, ExerciseAngina, ST_Slope), binary (FastingBS).
- At least three plots: class distribution bar chart, histograms of numeric features (the zero spike in Cholesterol will be obvious), and a categorical feature vs. target chart (e.g. ST_Slope).
- Data checks: missing values, dtypes, zeros, negative Oldpeak values, and any feature that looks like it leaks the label.
- `DummyClassifier(strategy="most_frequent")` baseline: show it reaches about 55% accuracy while being useless, and explain why accuracy alone is misleading.

3. Part 2 - Preprocessing

- Split **first**: `train_test_split(..., stratify=y, random_state=SEED)`. Lock the test set away.
- **Basic pipeline:** keep values as-is, SimpleImputer(median) for numeric, OneHotEncoder(handle_unknown="ignore") for categorical, inside a ColumnTransformer.
- **Improved pipeline:** replace zeros in Cholesterol and RestingBP with NaN, add missingness indicators (add_indicator=True), use KNNImputer or IterativeImputer (scale first for KNN). Justify why 0 means "not measured".
- **Fair comparison:** the same Random Forest with basic vs. improved pipeline, 3-fold CV on the training set only. This isolates the preprocessing effect.
- Talking point: zero-cholesterol rows correlate strongly with heart disease, likely due to how one source hospital recorded data. That is a site artifact, not biology.
- **Deliverable for teammates:** a function `build_preprocessor(kind="basic" | "improved")` plus X_train, X_test, y_train, y_test and the cv object.

Member 2: Modeling (Part 3, Part 4)

1. Part 3 - Random Forest vs. boosting

All models use Member 1's improved pipeline and the same cv object.

- **Random Forest:** GridSearchCV over 6 settings, e.g. max_depth in {None, 5, 10} x min_samples_leaf in {1, 5}.
- **XGBoost:** 6 settings, e.g. max_depth in {3, 5} x learning_rate in {0.05, 0.1, 0.3}.
- **AdaBoost:** default settings.
- **Dummy:** kept as the reference.
- Use return_train_score=True and save train and validation scores for every model (Member 3 needs them).
- Answer the guiding questions: boosting reduces bias by focusing on hard cases; it overfits with high learning rates or deep trees on small or noisy data; did extra complexity actually help?

2. Part 4 - Imbalance with SMOTENC

- Report the class ratio (about 508 positive vs. 410 negative).
- Build an `imblearn.pipeline.Pipeline` in this order: impute and scale numeric features, then SMOTENC(categorical_features=[...]) on the **un-one-hot-encoded** categoricals, then one-hot encode, then the model. SMOTENC cannot handle NaNs and should never see one-hot columns; scaling matters because SMOTE is distance-based.
- Compare the same model with and without SMOTENC in CV. Resampling then happens only inside training folds, never on the test set.

- Discuss whether it was needed: recall may rise while precision drops, and "no improvement on near-balanced data" is a valid result.

Member 3: Evaluation, explanation, integration (Parts 5, 6, 7)

1. Part 5 - Fair evaluation

- Comparison table of CV/validation accuracy, precision, recall, F1 and ROC-AUC for all models (positive class = 1), clearly labeled "validation".
- Choose the final model **from validation results only**.
- Train vs. validation gap table with an overfitting discussion (e.g. a deep RF with train score near 1.0).
- Then, **once**: fit on the full training set, evaluate on the test set, and show a confusion matrix plus a ROC or precision-recall curve.
- Answer: would we choose differently if false negatives were very costly? (e.g. pick by recall or lower the decision threshold.)

2. Part 6 - SHAP (bonus, +10)

- shap.TreeExplainer on the final model, with feature names from preprocessor.get_feature_names_out().
- Global: beeswarm plot. Local: waterfall plot for one false negative from the test set.
- Discuss whether the drivers are clinically sensible and why importance does not mean causation (the cholesterol-zero artifact is a perfect example).
- Optional (+5): LIME on the same patient, compared with SHAP.

3. Part 7 - Conclusions

250-400 words answering all six questions, using numbers from Members 1 and 2.

4. Integration and submission

- Merge everything into the master notebook and use Restart & Run All to confirm it runs top to bottom.
- Add a requirements / package versions cell and dataset download instructions.
- Citations: the Kaggle dataset plus at least two technical sources (scikit-learn, imbalanced-learn, XGBoost, SHAP docs).
- AI-assistance note and member contributions.

Suggested timeline

Days	What happens
1-2	Member 1 delivers the setup cell (split, CV object, build_preprocessor). Member 2 drafts model code; Member 3 drafts evaluation and plotting helpers.
3-5	Everyone runs their parts.
6	Member 3 merges and reruns the full notebook. Everyone reviews it.
7	Record the 5-minute video (about 1:40 per person). Each member explains their own section and what they learned. Member 1 explains the improved pipeline technique, since anything not covered in class must be explained in the video.

Note: the point totals in the task descriptions and the rubric table do not match (Evaluation 15 vs. 20, Conclusions 10 vs. 15). Ask the instructor which applies; the split above stays balanced either way.